

Data Binding

Angular needs a bridge between your TypeScript class and the HTML the user actually sees — and a way to react when the user interacts back. Four tools, four directions of data flow.

1 · INTERPOLATION

`{{ }}` — displaying data

```
name = 'Ahmed';

<h2>Hello {{ name }}</h2>
<!-- Result: Hello Ahmed -->
```

Use `{{ }}` whenever you just need to show a value — text, a number, or a simple expression. Real-world: user names & greetings, product titles, counters, status messages.

2 · PROPERTY BINDING

`[ ]` — binding to element properties

Problem: what if an attribute's value needs to come from a variable — like an image URL?

```
imageUrl = 'assets/avatar.png';

<img [src]="imageUrl">
```

The `[ ]` tells Angular "read this as a component property, not plain text." Works on any DOM property — `src`, `disabled`, `class`, and more. Real-world: images, disabling a button while loading, toggling a class.

3 · EVENT BINDING

`( )` — listening to user actions

Problem: how does Angular know the user clicked a button?

```
<button (click)="login()">Login</button>

login() { this.status = 'in'; }
```

User clicks

(click)



Component method

runs



UI update

re-renders

The `( )` means "listen for this event, then run this method." Works for any DOM event. Real-world: login buttons, add-to-cart, form submit, menu toggles.

4 · TWO-WAY BINDING

[()] — keeping data and UI in sync

Problem: an input field needs to update a variable, and show that variable's current value — at the same time.

```
username = '';  
  
<input [(ngModel)]="username">  
<!-- requires importing FormsModule -->
```

Two-way binding is really just [] and () combined — property binding in, event binding out. Real-world: search boxes, settings forms, live filters, profile editing.

5 · CHOOSING THE RIGHT BINDING

Rule of thumb

1

Data flows out

Use {{ }} or []

2

Data flows in

Something needs to flow back from the user — use ()

3

Both at once

Use [()]

6 · EXERCISE SOLUTION

A user profile component, using every tool

```
userName = 'Sara';  
imageUrl = 'assets/sara.png';  
status = 'Available';  
toggleStatus() { this.status = 'Busy'; }  
  
<h2>{{ userName }}</h2>  
<img [src]="imageUrl">  
<p>{{ status }}</p>  
<button (click)="toggleStatus()">Change Status</button>
```

- `userName` → `{{ }}` — just needs to be shown as text
- `imageUrl` → `[src]` — sets the `img` element's property
- `Change Status` → `(click)` — reacts to the user's click